

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 2
MODUL 12
“SEARCHING”



DISUSUN OLEH:
Andromeda Alike Ramadhani
109082500156
S1 IF-13-02

DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2024/2025

DASAR TEORI

Sequential Search atau pencarian berurutan adalah proses pencarian yang digunakan untuk mencari suatu data pada Kumpulan data secara berurutan. Cara kerjanya adalah dengan membandingkan data yang dicari dengan setiap elemen array secara berurutan, mulai dari indeks pertama sampai data ditemukan atau seluruh data selesai dicek. Metode ini tidak memiliki syarat sehingga dapat digunakan pada data yang sudah terurut maupun belum terurut.

Selanjutnya binary search adalah proses pencarian data dengan membagi data menjadi dua bagian secara terus menerus. Program akan mengambil nilai Tengah, lalu membandingkannya dengan data yang dicari. Jika data target lebih kecil dari nilai Tengah, pencarian dilakukan pada bagian kiri array. Sebaliknya, jika data target lebih besar, pencarian dilakukan pada bagian kanan array. Proses ini akan dilakukan berulang sampai data ditemukan atau data habis. Namun, metode pencarian ini hanya dapat digunakan pada data yang sudah dalam keadaan terurut.

Berdasarkan perbandingannya, sequential search lebih mudah digunakan karena tidak memerlukan data terurut, tetapi kurang cocok untuk diterapkan pada data dengan ukuran besar. Sementara binary search dapat melakukan pencarian lebih cepat, namun membutuhkan data yang sudah diurutkan terlebih dahulu.

CONTOH SOAL

1. Latihan1

Source Code:

```
package main

import (
    "fmt"
)

// Definisi struct mahasiswa sesuai modul
type mahasiswa struct {
    nama, nim, kelas, jurusan string
    ipk                          float64
}

// Definisi array of struct dengan kapasitas 2023
type arrMhs [2023]mahasiswa

// Fungsi Sequential Search: mencari berdasarkan NAMA
func SeqSearch_3(T arrMhs, n int, X string) int {
    var found int = -1
    var j int = 0
    for j < n && found == -1 {
        if T[j].nama == X {
            found = j
        }
        j = j + 1
    }
    return found
}
```

```

// Fungsi Binary Search: mencari berdasarkan NIM
(syarat: array HARUS terurut berdasarkan NIM)

func BinarySearch_3(T arrMhs, n int, X string) int {
    var found int = -1
    var med int
    var kr int = 0
    var kn int = n - 1

    for kr <= kn && found == -1 {
        med = (kr + kn) / 2
        if X < T[med].nim {
            kn = med - 1
        } else if X > T[med].nim {
            kr = med + 1
        } else {
            found = med
        }
    }

    return found
}

func main() {
    var dataMahasiswa arrMhs
    var n int = 4

    // Mengisi data dummy

    // PERHATIKAN: Data sengaja diurutkan dari NIM
    terkecil ke terbesar

    dataMahasiswa[0] = mahasiswa{nama: "Andi", nim:
    "1301210001", kelas: "IF-45-01", jurusan:
    "Informatika", ipk: 3.8}

```

```

        dataMahasiswa[1] = mahasiswa{nama: "Budi", nim:
"1301210005", kelas: "IF-45-01", jurusan:
"Informatika", ipk: 3.5}

        dataMahasiswa[2] = mahasiswa{nama: "Citra", nim:
"1301210010", kelas: "IF-45-02", jurusan:
"Informatika", ipk: 3.9}

        dataMahasiswa[3] = mahasiswa{nama: "Ren", nim:
"1301210020", kelas: "IF-45-02", jurusan:
"Informatika", ipk: 4.0}


        fmt.Println("=== Demo 1: Sequential Search (Cari
Nama) ===")

        targetNama := "Citra"

        idxNama := SeqSearch_3(dataMahasiswa, n,
targetNama)

        if idxNama != -1 {

            fmt.Printf("Ketemu! %s ada di indeks ke-%d\n",
targetNama, idxNama)

        } else {

            fmt.Println("Data tidak ditemukan.")

        }


        fmt.Println("\n=== Demo 2: Binary Search (Cari NIM)
===")

        targetNIM := "1301210020"

        idxNIM := BinarySearch_3(dataMahasiswa, n,
targetNIM)

        if idxNIM != -1 {

            fmt.Printf("Ketemu! NIM %s adalah milik %s di
indeks ke-%d\n", targetNIM, dataMahasiswa[idxNIM].nama,
idxNIM)

        } else {

            fmt.Println("Data tidak ditemukan.")

```

```
}  
  
}
```

The screenshot shows a Go IDE with several tabs. The active tab is 'Sequential & Binary - Struct Urut.go'. The code defines a struct for student data and implements sequential and binary search functions. A preview window on the right shows the output of the program, displaying student data for a specific NIM.

```
1 package main  
2  
3 import (  
4     "fmt"  
5 )  
6  
7 // Definisi struct mahasiswa sesuai modul  
8 type mahasiswa struct {  
9     nama, nim, kelas, jurusan string  
10    ipk float64  
11 }  
12  
13 // Definisi array of struct dengan kapasitas 2023  
14 type arrMhs [2023]mahasiswa  
15  
16 // Fungsi Sequential Search: mencari berdasarkan NAMA  
17 func SeqSearch_3(T arrMhs, n int, X string) int {  
18     var found int = -1  
19     var j int = 0  
20     for j < n && found == -1 {  
21         if T[j].nama == X {  
22             found = j  
23         }  
24         j = j + 1  
25     }  
26     return found  
27 }  
28  
29 // Fungsi Binary Search: mencari berdasarkan NIM (syarat: array HARUS terurut berdasarkan NIM)
```

Preview Window Output:

```
109082500156  
IF-13-02  
Andromeda Alika Ramadhani
```

Output:

The terminal window shows the execution of the program. It displays the results of two searches: a sequential search for a student named 'Citra' and a binary search for a student with NIM '1301210020'.

```
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\MODUL12> go run "e:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\MODUL12\Guided1\Sequential & Binary - Struct Urut.go"  
=== Demo 1: Sequential Search (Cari Nama) ===  
Ketemu! Citra ada di indeks ke-2  
  
=== Demo 2: Binary Search (Cari NIM) ===  
Ketemu! NIM 1301210020 adalah milik Ren di indeks ke-3  
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\MODUL12>
```

Deskripsi Program:

Program diatas adalah program yang digunakan untuk melakukan pencarian data mahasiswa. Pencarian data mahasiswa ini menggunakan dua cara yaitu sequential search dan binary search. Pertama kita simpan data mahasiswa ke dalam struct dengan nama mahasiswa yang berisi nama, nim, kelas, jurusan, dan ipk. Setelah itu, buat tipe data arrMhs untuk menampung 2023 data mahasiswa.

Fungsi SeqSearch_3 digunakan untuk mencari data mahasiswa berdasarkan nama menggunakan metode sequential search. Caranya adalah dengan memeriksa data satu per satu dari indeks pertama sampai data yang dicari ditemukan. Pertama kita inisialisasi variabel found dengan nilai -1 sebagai tanda bahwa data yang dicari belum ditemukan. Kemudian lakukan perulangan dari indeks pertama sampai terakhir. Jika

data yang dicari sudah ditemukan, maka perulangan akan dihentikan. Jika tidak maka perulangan akan dijalankan sampai akhir dan fungsi akan mengembalikan nilai -1.

Selanjutnya, fungsi `BinarySearch_3` yang digunakan untuk mencari data mahasiswa berdasarkan nim menggunakan metode binary search. Cara kerjanya adalah dengan membagi data menjadi dua bagian dan program menentukan indeks tengah kemudian membandingkan nilai NIM yang dicari dengan NIM yang ada pada posisi tengah. Jika nilai target yang dicari lebih kecil, maka pencarian akan dilanjutkan ke bagian kiri array. Sebaliknya, jika lebih besar maka pencarian berpindah ke bagian kanan array. Proses dilakukan sampai data yang dicari ditemukan. Penggunaan binary search ini harus menggunakan data yang sudah dalam kondisi terurut yaitu dari data terkecil sampai terbesar.

Terakhir kita buat func main, pertama kita buat variabel `dataMahasiswa` untuk menyimpan data mahasiswa. Setelah itu, program akan melakukan dua pencarian. Pertama dengan menggunakan sequential search untuk mencari mahasiswa dengan nama "Citra". Jika ditemukan, program akan menampilkan indeks tempat data berada. Kedua menggunakan binary search untuk mencari mahasiswa berdasarkan NIM. Jika data ditemukan, program akan menampilkan nama pemilik NIM beserta indeksnya. Jika tidak ditemukan, program akan menampilkan pesan "Data tidak ditemukan".

SOAL LATIHAN

Statement Searching

1. Soal1

Source Code:

```
package main

import "fmt"

const NMAX int = 21

type tabSuara [NMAX]int

func hitungSuara(calons *tabSuara, totalMasuk, suaraSah
*int) {
    var x int
    for {
        fmt.Scan(&x)
        if x == 0 {
            break
        }
        *totalMasuk += 1

        if x >= 1 && x <= 20 {
            *suaraSah += 1
            calons[x] += 1
        }
    }
}

func tampilkanHasil(calons tabSuara, totalMasuk,
suaraSah int) {
    var i int
```



```
        fmt.Println("Suara Masuk: ", totalMasuk)

        fmt.Println("Suara Sah: ", suaraSah)

        for i = 1; i <= 20; i++ {
            if calon[i] > 0 {
                fmt.Printf("%d: %d\n", i, calon[i])
            }
        }
    }

func main() {
    var calon tabSuara
    var suaraSah, totalMasuk int

    hitungSuara(&calon, &totalMasuk, &suaraSah)
    tampilkanHasil(calon, totalMasuk, suaraSah)
}
```

The screenshot shows a Go IDE with the file 'pilkart.go' open. The code defines a constant 'NMAX' as 21, a slice 'tabSuara' of type '[NMAX]int', and two functions: 'hitungSuara' and 'tampilkanHasil'. The 'hitungSuara' function uses a 'for' loop to read input with 'fmt.Scan(&x)'. It checks if 'x' is 0 to break the loop, increments 'totalMasuk' for any input, and increments 'suaraSah' and 'calon[x]' if 'x' is between 1 and 20. The 'tampilkanHasil' function prints 'totalMasuk' and 'suaraSah'. A browser window in the foreground shows the output: '109082500156', 'IF-13-02', and 'Andromeda Alike Ramadhani'.

```
1 package main
2
3 import "fmt"
4
5 const NMAX int = 21
6
7 type tabSuara [NMAX]int
8
9 func hitungSuara(calons *tabSuara, totalMasuk, suaraSah *int) {
10     var x int
11     for {
12         fmt.Scan(&x)
13         if x == 0 {
14             break
15         }
16         *totalMasuk += 1
17
18         if x >= 1 && x <= 20 {
19             *suaraSah += 1
20             calons[x] += 1
21         }
22     }
23 }
24
25 func tampilkanHasil(calons tabSuara, totalMasuk, suaraSah int) {
26     var i int
27     fmt.Println("Suara Masuk: ", totalMasuk)
28     fmt.Println("Suara Sah: ", suaraSah)
29 }
```

Output:

The terminal window shows the command 'go run "e:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\MODUL12\Soal11\pilkart.go"' being executed. The output shows the input '7 19 3 2 78 3 1 -3 18 19 0', followed by 'Suara Masuk: 10' and 'Suara Sah: 8'. Then, it lists the counts for each candidate index: '1: 1', '2: 1', '3: 2', '7: 1', '18: 1', and '19: 2'.

```
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\MODUL12> go run "e:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\MODUL12\Soal11\pilkart.go"
7 19 3 2 78 3 1 -3 18 19 0
Suara Masuk: 10
Suara Sah: 8
1: 1
2: 1
3: 2
7: 1
18: 1
19: 2
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\MODUL12>
```

Deskripsi Program:

Program diatas adalah program yang digunakan untuk menghitung hasil pemilihan ketua RT. Pertama kita deklarasikan konstanta NMAX dengan nilai 21 karena indeks array dimulai dari 0, sedangkan nomor calon hanya menggunakan angka 1-20. Kemudian kita buat tipe data tabSuara untuk menyimpan jumlah suara masing-masing calon. Selanjutnya kita buat fungsi hitungSuara yang digunakan untuk menerima input suara sekaligus menghitung hasil pemilihan. Pada fungsi ini kita gunakan perulangan for untuk menerima input satu per satu menggunakan fmt.Scan. Jika input yang diterima angka 0, maka proses input akan dihentikan dan setiap input selain 0 akan dihitung sebagai suara masuk, sehingga variabel totalMasuk bertambah satu. Selanjutnya, program akan melakukan pengecekan apakah nomor calon yang diinputkan berada pada rentang 1-20. Jika valid, maka suara dihitung sebagai suara sah dan jumlah suara akan ditambahkan pada array.

Selanjutnya kita buat fungsi tampilkanHasil yang digunakan untuk menampilkan hasil perhitungan suara ke layar. Pertama program akan mencetak jumlah suara yang masuk dan jumlah suara sah. Kemudian kita gunakan perulangan dari indeks

1 sampai 20 untuk mengecek apakah seorang calon mendapatkan suara. Jika jumlah suara lebih dari 0, maka jumlah suara akan ditampilkan sebagai output.

Terakhir pada fungsi main, kita buat variabel calon untuk menyimpan jumlah suara, variabel suaraSah dan totalMasuk untuk menyimpan hasil perhitungan. Selanjutnya, kita panggil fungsi hitungSuara untuk memproses data yang diinputkan dan panggil fungsi tampilkanHasil untuk menampilkan hasil perhitungan suara yang telah dilakukan.

2. Soal2

Source Code:

```
package main

import "fmt"

const NMAX int = 21

type tabSuara [NMAX]int

func hitungSuara(calon *tabSuara, totalMasuk, suaraSah
*int) {
    var x int
    for {
        fmt.Scan(&x)
        if x == 0 {
            break
        }
        *totalMasuk += 1

        if x >= 1 && x <= 20 {
            *suaraSah += 1
            calon[x] += 1
        }
    }
}
```

```

func cariPemenang(calon tabSuara, ketua, wakil *int) {
    var max1, max2, i int
    for i = 1; i <= 20; i++ {
        if calon[i] > max1 {
            max2 = max1
            *wakil = *ketua
            max1 = calon[i]
            *ketua = i
        } else if calon[i] == max1 && calon[i] > 0 {
            *wakil = i
        } else if calon[i] > max2 {
            max2 = calon[i]
            *wakil = i
        }
    }
}

```

```

func tampilkanHasil(totalMasuk, suaraSah, ketua, wakil
int) {
    fmt.Println("Suara Masuk: ", totalMasuk)
    fmt.Println("Suara Sah: ", suaraSah)
    fmt.Println("Ketua RT: ", ketua)
    fmt.Println("Wakil RT: ", wakil)
}

```

```

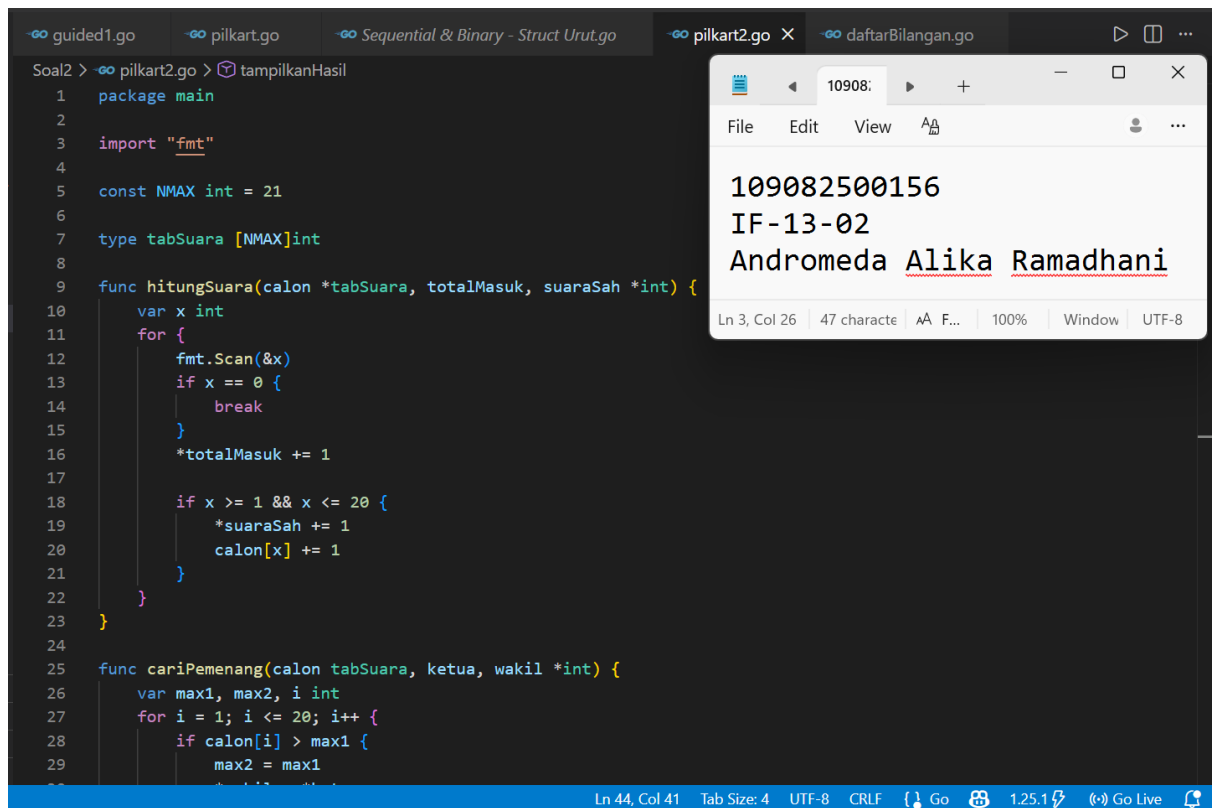
func main() {
    var calon tabSuara
    var suaraSah, totalMasuk, ketua, wakil int

    hitungSuara(&calon, &totalMasuk, &suaraSah)
    cariPemenang(calon, &ketua, &wakil)
}

```

```
tampilkanHasil(totalMasuk, suaraSah, ketua, wakil)

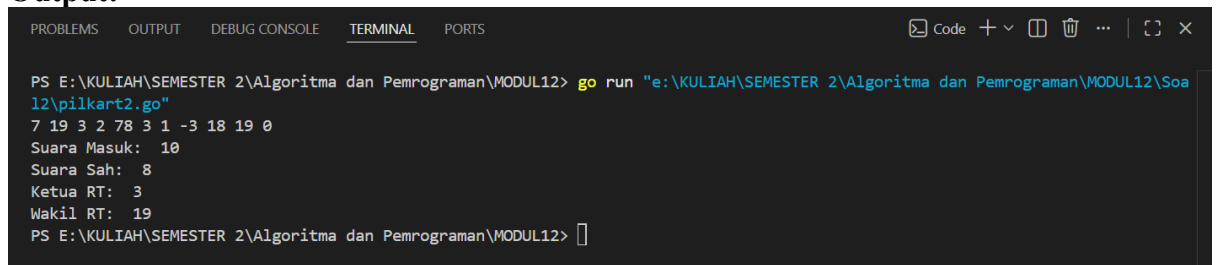
}
```



The screenshot shows a Go IDE with the file `pilkart2.go` open. The code defines a program to calculate election results. It includes a `hitungSuara` function that processes input votes and a `cariPemenang` function that determines the winner based on the number of votes. A browser window in the foreground displays the output of the program:

```
109082500156
IF-13-02
Andromeda Alike Ramadhani
```

Output:



The screenshot shows a terminal window with the following output:

```
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\MODUL12> go run "e:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\MODUL12\Soa
l2\pilkart2.go"
7 19 3 2 78 3 1 -3 18 19 0
Suara Masuk: 10
Suara Sah: 8
Ketua RT: 3
Wakil RT: 19
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\MODUL12> |
```

Deskripsi Program:

Program diatas adalah program yang digunakan untuk menghitung hasil pemilihan ketua RT dan menentukan siapa yang menjadi ketua RT dan wakil RT. Pertama kita deklarasikan konstanta NMAX dengan nilai 21 karena indeks array dimulai dari 0, sedangkan nomor calon hanya menggunakan angka 1-20. Kemudian kita buat tipe data `tabSuara` untuk menyimpan jumlah suara masing-masing calon. Selanjutnya kita buat fungsi `hitungSuara` yang digunakan untuk menerima input suara sekaligus menghitung hasil pemilihan. Pada fungsi ini kita gunakan perulangan `for` untuk menerima input satu per satu menggunakan `fmt.Scan`. Jika input yang diterima angka 0, maka proses input akan dihentikan dan setiap input selain 0 akan dihitung sebagai suara masuk, sehingga variabel `totalMasuk` bertambah satu. Selanjutnya, program akan melakukan pengecekan apakah nomor calon yang diinputkan berada pada rentang 1-20. Jika valid, maka suara dihitung sebagai suara sah dan jumlah suara akan ditambahkan pada array.

Selanjutnya kita buat fungsi cariPemenang yang digunakan untuk menentukan calon dengan suara terbanyak sebagai ketua RT dan suara terbanyak kedua sebagai wakil RT. Pertama kita buat variabel max1 untuk menyimpan jumlah suara tertinggi dan max2 untuk menyimpan jumlah suara tertinggi kedua. Selanjutnya program akan melakukan perulangan dari indeks 1 sampai 20 dan melakukan perbandingan menggunakan if-else. Jika ditemukan jumlah suara yang lebih besar dari max1, maka nilai max1 dipindahkan ke max2, posisi ketua dipindahkan menjadi wakil, kemudian posisi ketua diperbarui dengan calon yang memiliki suara lebih besar. Namun, jika ditemukan jumlah suara yang sama dengan suara max1, maka calon tersebut akan ditetapkan sebagai wakil.

Selanjutnya kita buat fungsi tampilkanHasil yang digunakan untuk menampilkan hasil perhitungan suara ke layar. Pertama program akan mencetak jumlah suara yang masuk dan jumlah suara sah, beserta nomor calon yang terpilih sebagai ketua RT dan wakil RT.

Terakhir pada fungsi main, kita buat variabel calon untuk menyimpan jumlah suara, variabel suaraSah dan totalMasuk untuk menyimpan hasil perhitungan dan variabel ketua dan wakil untuk menyimpan nomor calon pemenang. Selanjutnya, kita panggil fungsi hitungSuara untuk memproses data yang diinputkan dan panggil fungsi cariPemenang untuk menentukan pemenang. Terakhir, kita panggil fungsi tampilkanHasil untuk menampilkan hasil perhitungan suara dan pemenang pemilihan ketua RT yang telah dilakukan.

3. Soal3

Source Code:

```
package main

import "fmt"

const NMAX int = 1000000

var data [NMAX]int

func main() {
    var n, k, hasil int
    fmt.Scan(&n, &k)
    isiArray(n)
    hasil = posisi(n, k)
    if hasil == -1 {
        fmt.Println("TIDAK ADA")
    } else {
```

```
        fmt.Println(hasil)

    }

}

func isiArray(n int) {

    var i int

    for i = 0; i < n; i++ {

        fmt.Scan(&data[i])

    }

}

func posisi(n, k int) int {

    var kiri, kanan, tengah int

    kiri = 0

    kanan = n - 1

    for kiri <= kanan {

        tengah = (kiri + kanan) / 2

        if data[tengah] == k {

            return tengah

        } else if data[tengah] < k {

            kiri = tengah + 1

        } else {

            kanan = tengah - 1

        }

    }

    return -1

}
```

```
Soal3 > -go daftarBilangan.go > main
1 package main
2
3 import "fmt"
4
5 const NMAX int = 1000000
6
7 var data [NMAX]int
8
9 func main() {
10     var n, k, hasil int
11     fmt.Scan(&n, &k)
12     isiArray(n)
13     hasil = posisi(n, k)
14     if hasil == -1 {
15         fmt.Println("TIDAK ADA")
16     } else {
17         fmt.Println(hasil)
18     }
19 }
20
21 func isiArray(n int) {
22     var i int
23     for i = 0; i < n; i++ {
24         fmt.Scan(&data[i])
25     }
26 }
27
28 func posisi(n, k int) int {
29     var kiri, kanan, tengah int
```

10908: 109082500156
IF-13-02
Andromeda Alika Ramadhani

Ln 3, Col 26 | 47 character | A F... | 100% | Window | UTF-8

Ln 16, Col 13 | Tab Size: 4 | UTF-8 | CRLF | Go | 1.25.1 | Go Live

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\MODUL12> go run "e:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\MODUL12\Soal3\daftarBilangan.go"
12 534
1 3 8 16 32 132 323 323 534 543 823 999
8
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\MODUL12> go run "e:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\MODUL12\Soal3\daftarBilangan.go"
12 535
1 3 8 16 32 132 323 323 534 543 823 999
TIDAK ADA
PS E:\KULIAH\SEMESTER 2\Algoritma dan Pemrograman\MODUL12>
```

Deskripsi Program:

Program diatas adalah program yang digunakan untuk mencari apakah sebuah bilangan k terdapat di dalam kumpulan data yang telah terurut membesar. Jika bilangan tersebut ditemukan, program akan menampilkan indeks posisi data tersebut, sedangkan jika tidak ditemukan maka program akan menampilkan tulisan "TIDAK ADA". Pada kode program ini menggunakan metode binary search karena data sudah dalam kondisi terurut.

Pertama, kita deklarasikan konstanta NMAX dengan nilai 1000000 yang menandakan kapasitas maksimum data yang dapat disimpan. Selanjutnya kita deklarasikan variabel data yang berupa array integer. Pada fungsi main, pertama kita deklarasikan variabel n untuk menyimpan jumlah data yang akan diinputkan, variabel k untuk menerima input bilangan yang ingin dicari, dan variabel hasil. Selanjutnya, kita gunakan fmt.Scan untuk menerima input n dan k. Selanjutnya kita panggil fungsi isiArray(n) untuk mengisi array sebanyak n data yang sudah terurut membesar. Setelah itu, kita panggil fungsi posisi(n, k) untuk mencari posisi bilangan k pada array. Hasil pencariannya kemudian disimpan ke dalam variabel hasil. Jika nilai hasil sama dengan

-1, artinya data tidak ditemukan dan program akan menampilkan output “TIDAK ADA”. Sebaliknya, jika ditemukan, program akan mencetak indeks posisi bilangan tersebut.

Fungsi `isiArray()` berfungsi untuk membaca dan menyimpan data ke dalam array data. Proses pengisian dilakukan menggunakan perulangan `for` yang dimulai dari indeks 0 sampai `n`, kemudian kita diminta menginputkan setiap data ke dalam array.

Terakhir adalah fungsi `posisi()` yang digunakan untuk mencari data menggunakan binary search. Fungsi ini menerima `n` sebagai jumlah elemen dan `k` sebagai data yang dicari. Di dalam fungsi, kita deklarasikan lagi variabel `kiri`, `kanan`, dan `tengah`. Variabel `kiri` digunakan untuk menyimpan batas awal, `kanan` digunakan untuk menyimpan batas akhir, dan `tengah` untuk menentukan indeks tengah array. Proses pencarian dilakukan menggunakan perulangan selama nilai `kiri` masih lebih kecil atau sama dengan `kanan`. Program akan menghitung indeks tengah menggunakan rumus $(kiri+kanan)/2$. Jika nilai tengah sama dengan `k`, maka fungsi akan langsung mengembalikan indeks tersebut karena data telah ditemukan. Namun, jika nilai tengah lebih kecil dari `k`, berarti data yang dicari berada di sebelah kanan sehingga batas `kiri` dipindahkan menjadi `tengah+1`. Sebaliknya, jika nilai Tengah lebih besar dari `k`, maka pencarian akan dipindahkan ke sebelah kiri dan batas `kanan` diubah menjadi `tengah-1`. Proses ini akan terus dilakukan sampai data ditemukan. Jika pencarian selesai tetapi data tidak ditemukan, fungsi akan mengembalikan nilai -1.

DAFTAR PUSTAKA

Program Studi S1 Informatika. (2026). *Modul algoritma dan pemrograman 2 IF REG 2026*. Universitas Telkom.

Putra, F. P. (2020, 1 Maret). *Metode Binary Search dengan Golang*. Medium. <https://medium.com/@fajarpratamap/metode-binary-search-dengan-golang-427e9441ae8b>

Yasin, A. M. (2021, 15 Agustus). *Belajar Golang - Implementasi Algoritma Sequential Search*. Afrizals Blog. <https://afrizalmy.com/belajar-golang-implementasi-algoritma-sequential-search>